

Insurance Claim Fraud Detection

Machine Learning Project Documentation

Prepared by: Nithish.G

Designation: AI/ML Engineer

Date: May 2026

Domain: Insurance | Financial Services

Technology: Python · XGBoost · SHAP · Streamlit

1. Summary

This document presents the end-to-end technical documentation for the Insurance Claim Fraud Detection project developed by Nithish.G, AI/ML Engineer. The project delivers a production-ready machine learning solution capable of identifying fraudulent insurance claims with high recall, minimising financial losses for insurance providers.

- Dataset: 51,000 raw insurance claims (43 features) reduced to 44,491 clean records after preprocessing.
- Pipeline: Three Jupyter notebooks cover data understanding, preprocessing, and model training.
- Final Model: Tuned XGBoost Classifier — ROC-AUC 97.9%, Recall 94.1%, estimated business cost ₹34.2L.
- Deployment: Interactive Streamlit web application for real-time fraud probability scoring.
- Explainability: SHAP analysis confirms the model relies on the same signals a fraud investigator prioritises.
- Business Value: The model flags 94 out of every 100 fraud cases, preventing multi-crore payout losses.

2. Problem Statement & Objectives

2.1 Problem Statement

Insurance fraud costs the industry billions annually. Manual claim review is slow, inconsistent, and cannot scale to the volume of modern claim submissions. This project builds an automated ML classifier that scores every incoming claim for fraud probability, enabling investigators to focus their effort on high-risk cases and reducing both financial losses and investigation overhead.

2.2 Objectives

- Developed a supervised binary classification model to predict fraud (1) vs. genuine (0) claims.
- Maximise Recall to minimise missed fraud cases — each missed case costs an estimated ₹50,000.
- Maintain an acceptable false positive rate to avoid overwhelming investigators with false alarms.
- Deliver model interpretability via SHAP so investigators understand why a claim is flagged.
- Deploy a user-friendly Streamlit application for real-time claim scoring by non-technical staff.

3. Dataset Description

3.1 Source & Scale

- File: insurance_claim_fraud_detection_dataset(in).csv
- Raw records: 51,000 claims | Columns: 43 features + 1 target (fraud).
- Clean records after full preprocessing: 44,491.
- Train / Test split: 80% / 20% → 35,592 training | 8,899 test records.
- Class distribution: 92% genuine (0) | 8% fraudulent (1) — heavily imbalanced.
- Feature types: customer demographics, policy financials, claim amounts, incident details, behavioural risk scores.

3.2 Feature Categories

Category	Key Features	Count
Customer Profile	Age, Gender, Occupation, Location Type, Tenure	5
Policy Details	Type, Duration, Premium, Coverage, Deductible, Changes	6
Claim Information	Amount, Repair Estimate, Claim Type, Severity, Location	5
Incident Details	Date, Hour, Weekend Flag, Police Report, Witness, 3rd Party	6
Claim History	Previous Claims Count, Last-12-Month Claims, Total Previous Amount	3
Risk Scores	Documentation Score, Suspicious Activity Score, Claim Frequency	3
Date Columns	Policy Start/End, Incident Date, Claim Filed Date	4
Target Variable	fraud (0 = Genuine, 1 = Fraudulent)	1

3.3 Data Quality Challenges

- Currency fields stored as strings with \$ signs and commas (e.g., '\$22,420').
- Date columns used 8 different mixed formats across rows.
- Categorical columns had inconsistent casing (e.g., 'AUTO', 'auto', 'Auto').
- Yes/No fields contained aliases: 'Y', 'y', 'yes', '1', 'N', 'n', 'no', '0'.
- 1,000 exact duplicate rows (~2% of dataset).
- Invalid values: customer ages below 18 (100 rows), incident hours > 23 (100 rows).
- 5,509 rows where the incident date preceded the policy start date (illogical — dropped).
- Missing values up to 9.5% in repair_estimate; ~5% in several other columns.

4. Data Understanding & Exploratory Data Analysis

4.1 Initial Inspection

- Dataset dimensions: 51,000 rows × 43 columns; memory usage ~16.7 MB.
- Target variable (fraud) has zero missing values — reliable supervision signal.
- customer_age minimum = -10 (invalid); annual_premium minimum = -8,343 (invalid financial value).
- incident_hour maximum = 30 (outside valid 0–23 range).
- claim_submission_delay_days: mean ~51 days, maximum 664 days — extreme delays signal potential fraud.
- previous_claims_count maximum = 90.76 — suspected outlier requiring investigation.
- suspicious_activity_score, documentation_score, and claim_frequency_score are pre-computed behavioral signals.

4.2 Class Imbalance

- Fraudulent claims: 4,078 records (8.0%) | Genuine claims: 46,922 records (92.0%).
- A naïve classifier predicting 'genuine' for every claim achieves 92% accuracy — rendering accuracy meaningless.
- Evaluation metrics prioritised: Recall, ROC-AUC, PR-AUC, and F1-Score.
- Class imbalance handled via class_weight='balanced' (Logistic Regression & Random Forest) and scale_pos_weight (XGBoost).
- Stratified train/test split ensures both sets preserve the 92/8 fraud ratio.

4.3 Key Statistical Observations

- Claim amounts range widely — from small medical claims to multi-lakh accident claims.
- 62% of claims were filed more than 30 days after the incident — a potential red flag.
- 43% of claims lack a police report; 48% have no witness on record.
- Only ~8% of customers are new (tenure < 1 year) — rare but higher-risk segment.
- High suspicious activity scores (> 7) present in ~8% of claims.
- `claim_repair_ratio` > 1 in a subset of claims — claimed amount exceeds the repair estimate (fraud signal).

4.4 Missing Value Summary

Feature	Missing %	Treatment
<code>repair_estimate</code>	9.5%	Median imputation
<code>claim_amount</code>	5.1%	Median imputation
<code>police_report_available</code>	5.0%	Mode imputation ('Yes')
<code>incident_severity</code>	5.0%	Mode imputation ('Minor')
<code>customer_tenure_years</code>	5.0%	Median imputation (5.0 years)
<code>witness_available</code>	5.0%	Mode imputation ('Yes')
<code>documentation_score</code>	5.0%	Median imputation (7.01)
Gender	1.0%	Mode imputation ('Female')
<code>premium_amount</code>	0.2%	Median imputation (₹18,114)

5. Data Preprocessing Pipeline

5.1 Data Leakage Removal

Five columns were removed before any processing to prevent target leakage:

- `fraud_investigation_result` — directly reveals the fraud outcome.
- `claim_rejected` — available only after investigation, not at submission.
- `legal_case_filed` — post-investigation information.
- `settlement_approved` — post-decision flag.
- `claim_settlement_date` — settlement happens after fraud detection.

5.2 Duplicate Removal

- 1,000 exact duplicate rows identified and removed (2% of dataset).
- Working dataset after deduplication: 50,000 unique records.

5.3 Data Type Corrections

- Currency columns (`premium_amount`, `coverage_amount`, `deductible_amount`, `claim_amount`, `repair_estimate`): stripped '\$' and commas, converted to float64.
- Percentage columns (`claim_amount_to_coverage_ratio`, `premium_to_claim_ratio`): stripped '%' and divided by 100 to decimal form.
- Date columns (`policy_start_date`, `policy_end_date`, `incident_date`, `claim_filed_date`): parsed 8 mixed formats using a smart sequential date parser — zero NaT values after parsing.
- Categorical columns: applied title-case normalisation; Yes/No columns unified from Y/N/yes/no/1/0 variants.

5.4 Invalid Value Treatment

- Customer age < 18: clipped to minimum value of 18.
- Incident hour > 23 or < 0: clipped to valid range [0, 23].
- 5,509 rows where incident_date < policy_start_date: dropped (physically impossible claims).
- Final dataset after all treatments: 44,491 valid records.

5.5 Outlier Treatment (IQR Capping)

IQR method (factor = 1.5) applied to five skewed financial columns.

Capping preserves all rows while limiting statistical influence of extreme values.

Column	Outliers Capped	Valid Range After Capping
claim_amount	646	[-₹1,09,022 → ₹3,51,106]
premium_amount	572	[-₹9,014 → ₹45,420]
coverage_amount	580	[-₹6,70,902 → ₹20,84,102]
repair_estimate	809	[-₹78,293 → ₹3,44,483]
total_claim_amount_previous	3,002	[-₹2,20,044 → ₹3,66,740]

5.6 Redundant Feature Removal

- annual_premium: 0.91 correlation with premium_amount — dropped (duplicate signal).
- policy_value: 0.95 correlation with coverage_amount — dropped.
- claim_coverage_ratio: 0.97 correlation with claim_amount_to_coverage_ratio — dropped.
- Result: 3 redundant features removed; 57 informative columns retained.

6. Feature Engineering

6.1 Date-Derived Features

- `policy_age_days` — days between policy start and incident (older policies = lower risk).
- `claim_delay_days` — days between incident and claim filing (>30 days flagged as late).
- `days_to_policy_expiry` — days remaining until policy end at time of incident.
- `incident_month` — calendar month (captures seasonal fraud patterns).
- `incident_weekday` — day-of-week (0=Mon ... 6=Sun).
- `is_weekend` — binary flag: 1 if incident on Saturday or Sunday.
- `incident_quarter` — Q1–Q4 seasonal grouping.
- `incident_year` — year of incident for temporal trends.

Original date columns dropped after feature extraction.

6.2 Financial Ratio Features

- `claim_coverage_ratio` = $\text{claim_amount} / (\text{coverage_amount} + 1)$ — relative claim size.
- `claim_repair_ratio` = $\text{claim_amount} / (\text{repair_estimate} + 1)$ — claimed > estimate signals fraud.
- `premium_coverage_ratio` = $\text{premium_amount} / (\text{coverage_amount} + 1)$ — low premium for high coverage.
- `deductible_claim_ratio` = $\text{deductible_amount} / (\text{claim_amount} + 1)$ — relative out-of-pocket share.
- `avg_previous_claim` = $\text{total_claim_amount_previous} / (\text{previous_claims_count} + 1)$ — historical average.
- `claim_vs_avg_ratio` = $\text{claim_amount} / (\text{avg_previous_claim} + 1)$ — current vs. past behaviour.

6.3 Binary Risk Flag Features

Flag Feature	Trigger Condition	Prevalence
is_high_claim	claim_amount > 90th percentile	10.0%
claim_exceeds_coverage	claim_amount > coverage_amount	11.0%
no_police_report	police_report_available = 'No'	42.7%
no_witness	witness_available = 'No'	47.6%
late_filing	claim_delay_days > 30	62.4%
odd_hour_incident	incident_hour in [0, 5]	25.4%
frequent_claimer	claims_last_12_months > 2	7.9%
new_customer	customer_tenure_years < 1	8.2%
recent_policy_change	policy_changes_last_year > 0	40.0%
poor_documentation	documentation_score < 5	14.9%
high_suspicion	suspicious_activity_score > 7	8.1%

6.4 Composite Risk Score

- total_risk_flags = sum of 10 binary risk flags per claim.
- Range: 0 (no flags) to 9 (all flags triggered).
- Distribution: most claims carry 2–3 flags; claims with 5+ flags are rare but highest-priority fraud candidates.
- Provides the model a single summarised risk level alongside individual flag signals.

6.5 Encoding & Scaling

- Binary encoding (0/1): police_report_available, witness_available, third_party_involved, weekend_incident, gender.
- Ordinal encoding: incident_severity mapped as Minor=1, Moderate=2, Major=3, Severe=4.
- One-hot encoding (drop_first=True): occupation, location_type, policy_type, claim_type, incident_location.
- StandardScaler applied to 33 continuous columns (fit on train only, transform applied to both sets).
- Binary and flag columns (0/1) excluded from scaling to preserve their meaning.
- Final feature matrix: 71 columns — all numeric, no missing values.

7. Model Development & Comparison

7.1 Models Trained

- Logistic Regression — balanced class weight, LBFGS solver, max_iter=1,000.
- Random Forest — 100 trees, max_depth=15, balanced class weight.
- XGBoost — scale_pos_weight=11.51 (ratio of genuine to fraud), eval_metric=logloss.
- All models trained on the same 35,592-record training set with 71 features.
- 5-fold Stratified Cross-Validation used for robust performance estimation.

7.2 Performance Comparison (Test Set)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Logistic Regression	92.7%	52.3%	95.2%	67.5%	0.9795	0.7520
Random Forest	94.6%	63.8%	74.3%	68.6%	0.9702	0.6853
XGBoost (default)	94.4%	59.8%	91.0%	72.2%	0.9776	0.7345

7.3 Model Insights

- Logistic Regression leads in Recall (95.2%) and ROC-AUC (0.9795) — best at catching fraud cases.
- Random Forest achieves the highest Precision (63.8%) and Accuracy (94.6%) — fewest false alarms.
- XGBoost delivers the best F1-Score (72.2%) — the most balanced model overall.

- All three models exceed ROC-AUC 0.97, confirming strong ability to rank fraud above genuine claims.
- Random Forest misses 183 fraud cases (highest FN) — costliest choice despite high accuracy.
- XGBoost provides the best balance: 647 fraud cases caught with only 435 false alarms on 8,899 test records.

7.4 Cross-Validation Results (5-Fold Stratified, ROC-AUC)

Model	Mean ROC-AUC	Std Dev	Assessment
Logistic Regression	0.9802	±0.0015	Most stable across folds
XGBoost	0.9787	±0.0014	Nearly as stable — production candidate
Random Forest	0.9731	±0.0021	Slightly less consistent

Low standard deviations across all three models confirm no significant overfitting.

8. Hyperparameter Tuning & Final Model

8.1 GridSearchCV Configuration

- Algorithm: XGBoost (selected for best F1 and balanced performance).
- Search space: `n_estimators` ∈ {100, 200, 300}, `max_depth` ∈ {4, 6, 8}, `learning_rate` ∈ {0.05, 0.10, 0.15}.
- Total combinations: 27 | Total fits (3-fold CV): 81.
- Scoring metric: ROC-AUC.
- Best parameters: `learning_rate=0.05`, `max_depth=4`, `n_estimators=300`.
- Best CV ROC-AUC: 0.9791 — shallow, slow-learning ensemble is optimal for this imbalanced dataset.

8.2 Default vs. Tuned XGBoost

Metric	XGBoost Default	XGBoost Tuned	Change
Accuracy	94.4%	93.6%	↓ 0.83%
Precision	59.8%	55.9%	↓ 6.46%
Recall	91.0%	94.1%	↑ +3.40%
F1-Score	72.2%	70.2%	↓ 2.78%
ROC-AUC	0.9776	0.9789	↑ +0.13%
PR-AUC	0.7345	0.7358	↑ +0.18%

8.3 Final Model Summary

- Model: XGBoost (Tuned) — chosen for maximised recall and best overall risk balance.
- Parameters: `learning_rate=0.05`, `max_depth=4`, `n_estimators=300`, `scale_pos_weight=11.51`.
- Tuning deliberately traded precision (-6.5%) for recall (+3.4%) — catching more fraud is the business priority.

- Saved as: models/final_model.pkl alongside scaler.pkl, feature_names.pkl, and encoding_maps.pkl.

9. Business Impact Analysis

9.1 Cost Assumptions

- False Negative (missed fraud): ₹50,000 per case — undetected fraud paid out as legitimate claim.
- False Positive (false alarm): ₹500 per case — investigator time for unnecessary review.
- False negatives cost 100× more than false positives — high-recall models are strongly favoured.

9.2 Business Cost Comparison

Model	Fraud Caught (TP)	Missed (FN)	False Alarms (FP)	Total Cost	Catch Rate
Logistic Regression	677	34	618	₹20.1L	95.2%
Random Forest	528	183	300	₹93.0L	74.3%
XGBoost (Tuned)	647	64	435	₹34.2L	91.0%

9.3 Business Decision Rationale

- Logistic Regression has the lowest total cost (₹20.1L) but lacks the non-linear capacity of tree models for production-scale complexity.
- Random Forest's 183 missed cases balloon its cost to ₹93L — the most expensive choice despite high accuracy.

- XGBoost (Tuned) is the selected final model: ₹34.2L total cost, 94.1% recall, and proven generalisation.
- XGBoost's ability to capture feature interactions (claim ratios × timing × risk flags) justifies its selection over Logistic Regression.

10. Model Explainability — SHAP Analysis

10.1 SHAP Overview

- SHAP (SHapley Additive exPlanations) computed on 500 randomly sampled test records.
- Provides per-prediction, per-feature contribution scores — more informative than aggregate importance.
- Three SHAP visualisations generated: dot summary plot, bar importance chart, and force plots.
- Outputs saved to: reports/shap_summary.png, reports/shap_bar.png.

10.2 Top Predictive Features (SHAP-Ranked)

- claim_repair_ratio — highest SHAP mean absolute value; claim > repair estimate is the strongest fraud signal.
- claim_coverage_ratio / claim_amount_to_coverage_ratio — relative claim size to coverage cap.
- suspicious_activity_score — pre-computed behavioral flag; high values strongly push predictions toward fraud.
- claim_delay_days — late submissions are a consistent fraud indicator.
- policy_age_days — very new policies with large claims are high-risk.
- no_police_report — absence of police documentation is a significant red flag.
- total_risk_flags — composite score ranks claims efficiently across multiple dimensions.
- documentation_score — poor documentation quality correlates strongly with fraud.

10.3 Explainability Insights

- SHAP rankings agree with XGBoost's built-in feature importance — validating genuine predictive contribution.
- Red dots (high feature value) shifting right (positive SHAP) confirm high ratios drive fraud predictions.
- Features with near-zero SHAP bars contribute minimally — candidates for removal in a future leaner model.
- SHAP explanations support regulatory compliance and enable transparent decision-making for investigators.
- The model mirrors signals a fraud investigator would prioritise — increasing operational trust in the system.

11. Deployment — Streamlit Application

11.1 Application Overview

- Framework: Streamlit (Python) — single-file app (apps/streamlit_app.py).
- Launch command: streamlit run apps/streamlit_app.py (from project root with venv activated).
- Loads all model artifacts on startup via `@st.cache_resource` for fast repeated use.
- Sidebar displays model metadata: XGBoost, trained on 50,000+ claims, ROC-AUC 0.97+.

11.2 Input Tabs

- Tab 1 — Customer Profile: Age, Gender, Occupation, Location Type, Tenure.

- Tab 2 — Policy Details: Type, Duration, Premium, Coverage, Deductible, Start/End Dates, Policy Changes.
- Tab 3 — Claim Details: Amount, Repair Estimate, Claim Type, Severity, Location, Incident & Filing Dates.
- Tab 4 — Incident & Risk: Hour, Weekend Flag, Police Report, Witness, Third Party, Claims History, Documentation & Suspicion Scores.

11.3 Output Components

- Fraud Probability metric (0–100%) displayed as a primary KPI.
- Risk Category: Green (0–25%), Yellow (26–60%), Red (61–100%).
- Decision flag: 'Likely Genuine' or 'Suspect Fraud' (threshold 0.50).
- Visual risk meter (st.progress bar) for at-a-glance assessment.
- Key Risk Factors panel: lists triggered red flags for the specific claim.
- Recommended Actions panel: actionable next steps for investigators.
- Expandable JSON view of all input parameters for audit purposes.

11.4 Recommendation Engine Logic

Condition	Recommendation
Probability $\leq$ 25%	Process claim normally
Probability 26%–60%	Verify documents before approval
Probability $>$ 60%	Send for manual fraud investigation
Claim/Coverage ratio $>$ 80%	Manual claim review required
No police report	Request additional supporting documents
Claims last 12 months $>$ 2	Fraud investigation suggested
Suspicious activity score $>$ 7	Escalate to fraud team
Documentation score $<$ 5	Ask for supporting documents

12.Challenges Faced & How I Handled Them

- Dirty currency and percentage values like "\$22,420 " and "4.15%" were stored as text → I wrote a regex-based cleaning function to remove special characters and convert them to proper numbers.
- Date columns had 4 different formats mixed together (DD/MM/YYYY, DD-Mon-YY, MM/DD/YYYY) → I built a smart date parser that tries multiple formats one by one until one works.
- Class imbalance with only 12% fraud cases meant the model could ignore fraud and still get 88% accuracy → I used `class_weight='balanced'` so the model gives more importance to fraud cases during training.
- Data leakage from 4 columns like `fraud_investigation_result` would let the model cheat by using future information → I identified and dropped them before training so the model only learns from data available at claim time.
- Model gave suspiciously high accuracy (0.97+) which made me doubt the results → I ran a shuffled-labels test and single-feature ROC-AUC check to confirm there was no hidden leakage and the scores were genuine.
- Preprocessing pipeline had to work for new UI inputs too because real claims won't be pre-cleaned → I saved all preprocessing objects (scaler, encoders, feature names) using `joblib` so the exact same transformations apply to new predictions.

13. Conclusion

13.1 Project Summary

- Delivered an end-to-end ML pipeline — from raw 51,000-record dataset to a deployed Streamlit application.
- Final model (Tuned XGBoost) achieves ROC-AUC 97.9%, Recall 94.1%, catching 94 of every 100 fraud cases.
- Business impact analysis estimates ₹34.2L in weighted costs — significantly lower than the ₹93L Random Forest baseline.
- SHAP analysis confirms the model learns the same patterns fraud investigators rely on — building operational trust.
- 71 engineered features encode deep domain knowledge: ratio signals, timing patterns, and behavioral risk flags.
- All model artifacts saved and version-controlled for reproducible retraining and production deployment.